Okanagan College COSC 416 April 2026

SciKey Project

By: Brandon, James, John

SciKey Project Sponsors

Our team has selected the SciKey project that has been provided by members of The Paris East University

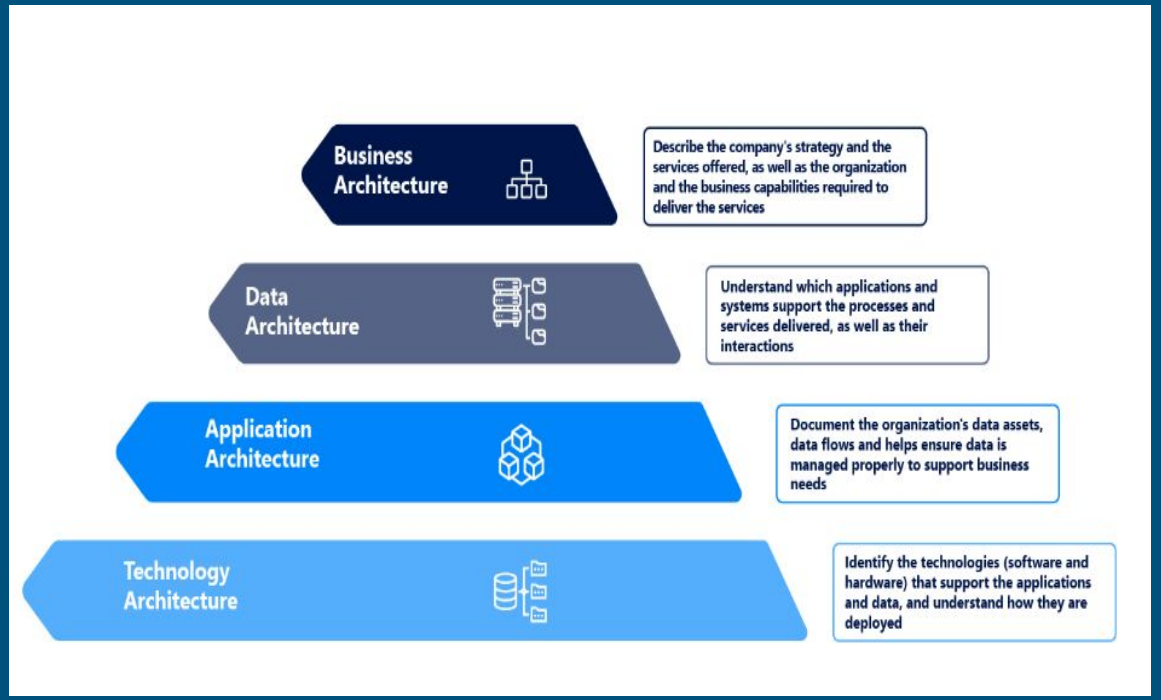
- Gaétan Hains (Prof. Comp. Sc.)
- Hélène Pipet (Documentary IT Manager)
- Jean Bouyssou (Documentary IT Scientist)

Project Overview

This projects goal is to use AI to automatically generate keywords by analyzing titles and abstracts from scientific publications from HAL open archive. The system will suggest relevant keywords for each publication and align them with existing identifiers from Wikidata. The generated keywords will be validated by comparing them against the ScanR website to calculate confidence rates. Eventually a web interface will allow users to query author information and export results

Technical Architecture

- Oracle 19c
- OLAP / Data Warehouse Hybrid
- Hardware Context
- Data Protection



Physical Architecture

System Resources

- 60% of Total RAM
- 80 / 20 SGA PGA Split
- Huge Pages

IO Optimization

- 2x Multiplexed Redo & Control Files
- Separate Tablespaces for Data & Indexes

Project Scope & Data Source

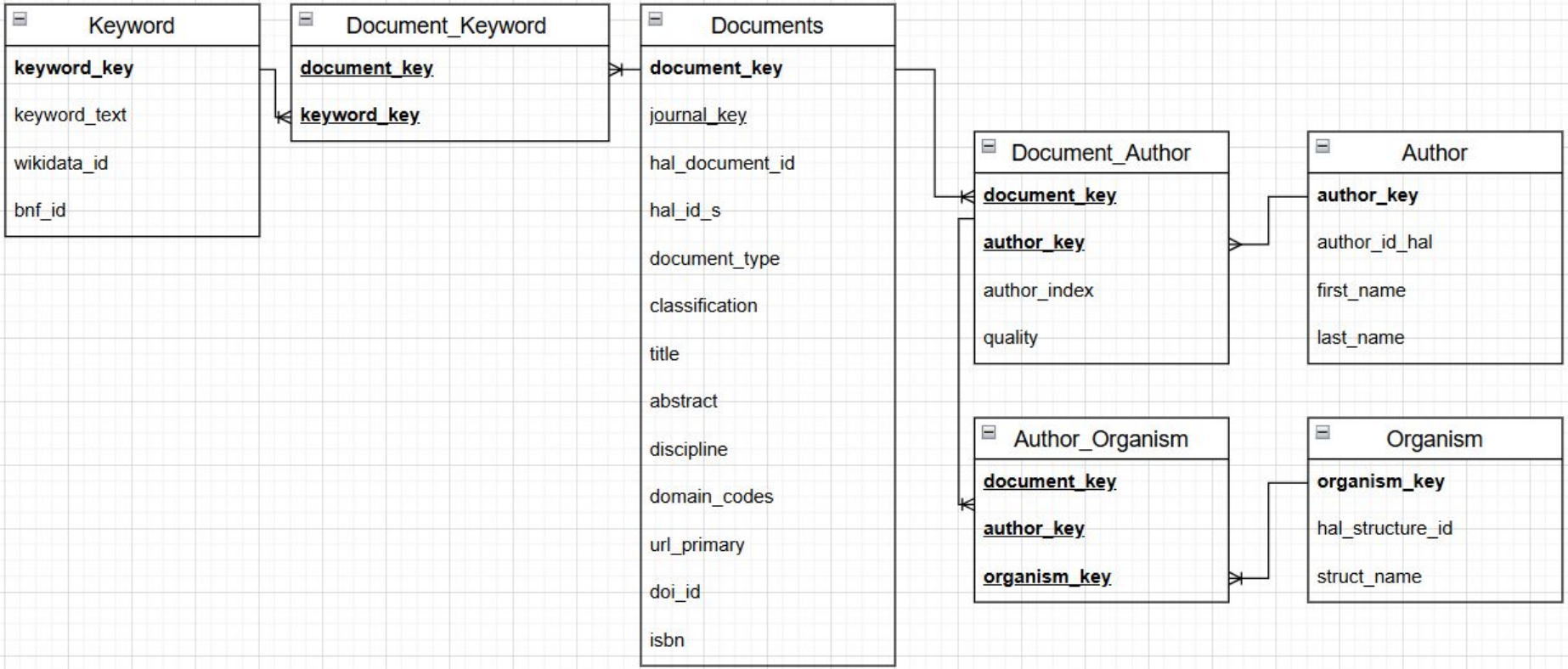
Scope

- Final general use database

Data Source

- HAL Open Archive
- UPEC Branch

ERD


Keyword
keyword_key
keyword_text
wikidata_id
bnf_id
Document_Keyword
document_key
keyword_key
Documents
document_key
journal_key
hal_document_id
hal_id_s
document_type
classification
title
abstract
discipline
domain_codes
url_primary
doi_id
isbn
Document_Author
document_key
author_key
author_index
quality
Author
author_key
author_id_hal
first_name
last_name
Author_Organism
document_key
author_key
organism_key
Organism
organism_key
hal_structure_id
struct_name

Performance Bottleneck

- DB CPU Time
- Parse Time Elapsed
- Physical Reads Per Second

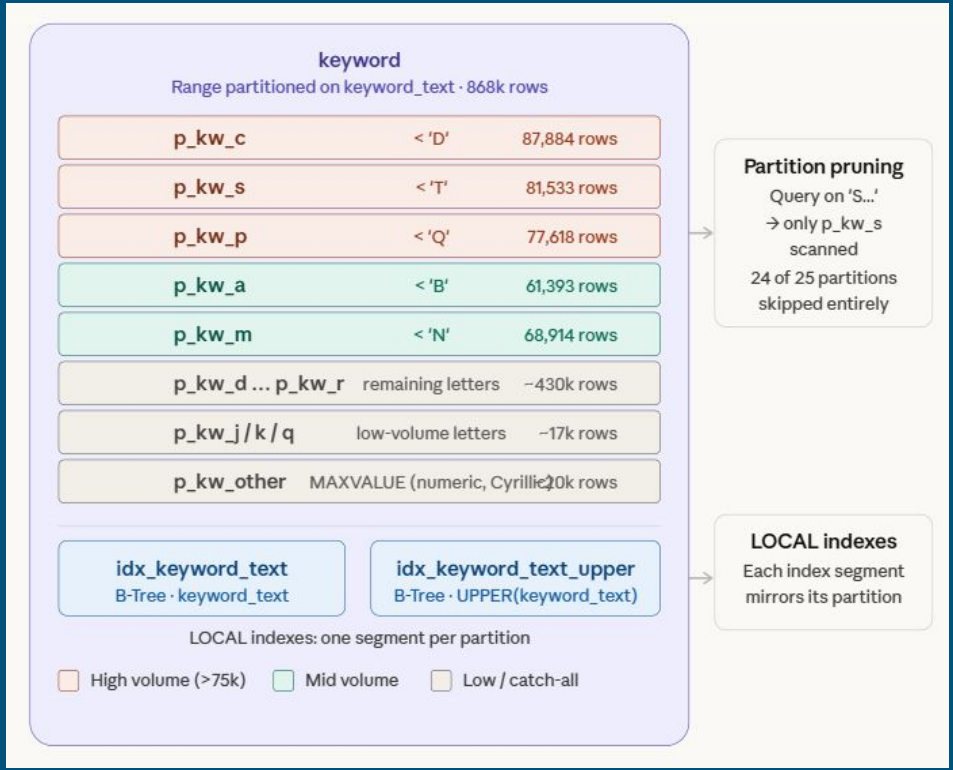
Partitioning and Indexes

Partitioning

- Evenly Distributed Partitions

Indexes

- Bitmap & B-Tree
- Local
- Non-Clustered



Materialized Views

- Based on ERD relationships (Document, Author, Keyword)
- Many-to-many relationships use composite keys
- Complex joins increase query cost
- Materialized views precompute joins
- Reduce repeated join operations
- Improve performance for read-heavy queries

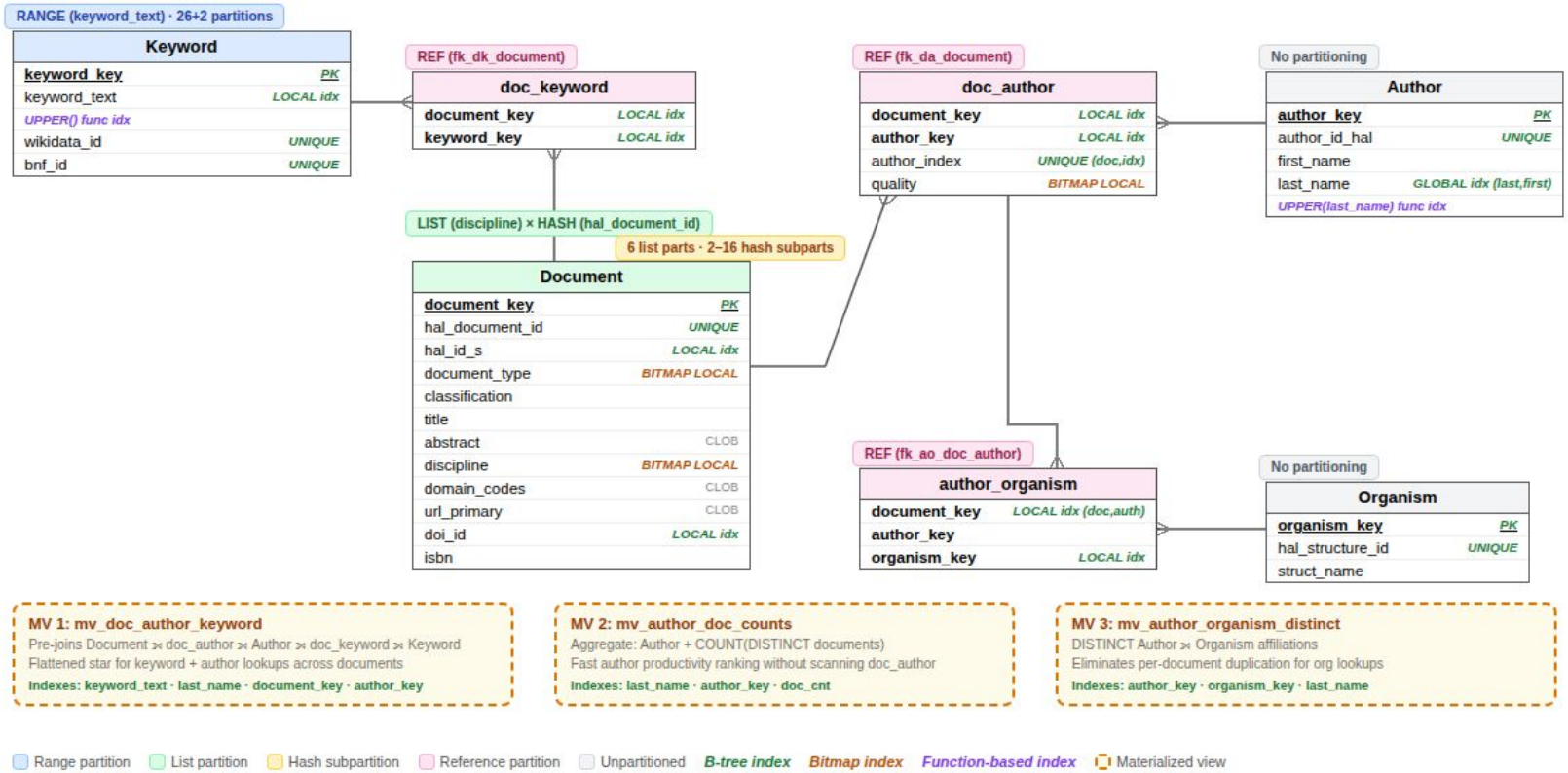
Materialized Views Comparison

General | Report

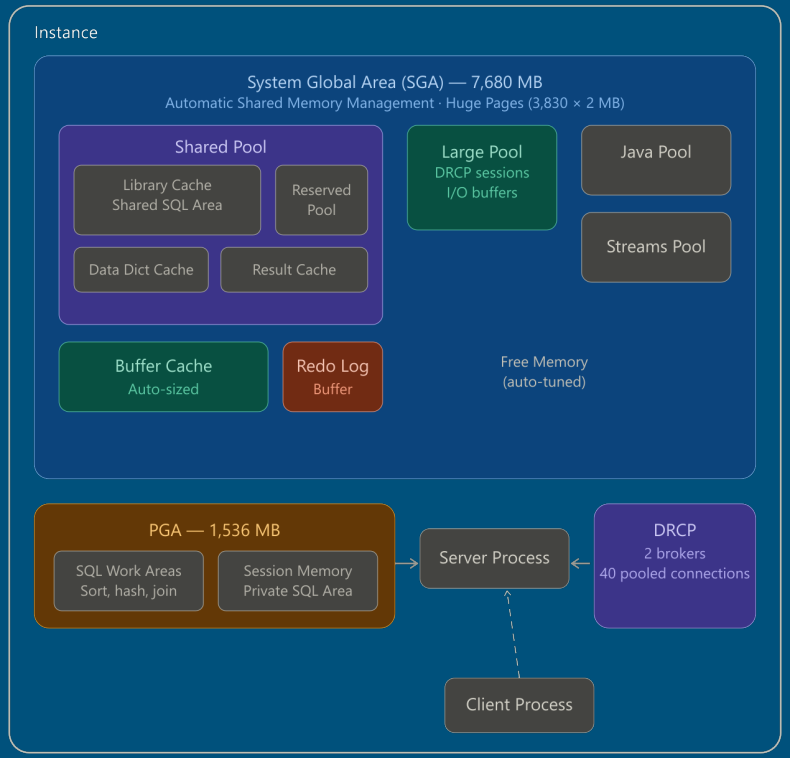
View Data: Per Second

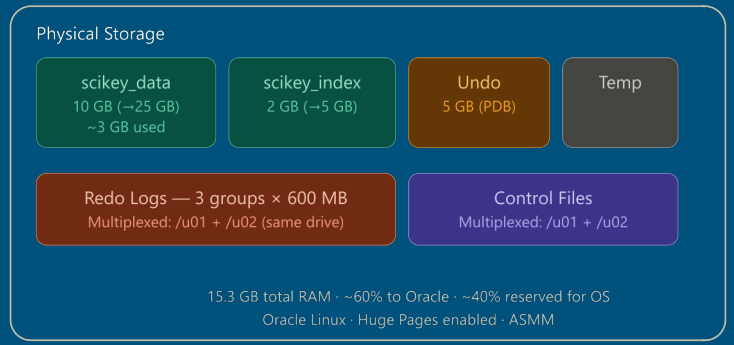
Name	First Period Metric Ratio	Second Period Metric Ratio	First Period Value	Second Period Value
DB cpu (seconds)			0.00	0.00
DB time (seconds)			11,224.84	6,473.70
db block changes			54,028.00	28,334.00
execute count			30,619,610.00	24,039,766.00
global cache cr block receive time (seconds)			0.00	0.00
global cache cr blocks received			0.00	0.00
global cache current block receive time (seconds)			0.00	0.00
global cache current blocks received			0.00	0.00
global cache get time (seconds)			0.00	0.00
global cache gets			0.00	0.00
opened cursors cumulative			30,642,932.00	24,051,822.00
parse count (total)			30,618,590.00	24,037,842.00
parse time cpu (seconds)			92.78	62.30
parse time elapsed (seconds)			2,235.84	1,900.38
physical reads			2,756.00	0.00
physical writes			0.00	74.00
redo size (KB)			10,219.02	8,102.63
session cursor cache hits			30,515,638.00	23,986,070.00
session logical reads			333,660.00	166,180.00

Design Diagram



Advanced Architecture & Design





The Tuning Methodology

- Data Integrity Risks
  - Stale data in materialized views
- Performance Risks
  - Query plan regression
  - High Cpu / wait events
- Mitigation
  - Refresh materialized views
  - Use workload validation script
  - Monitor with AWR and OEM

The Tuning Methodology

- Before:
  - Complex multi-table joins
  - High execution time
- After:
  - Queries use materialized views
  - Reduced join operations
- Techniques Used:
  - Materialized Views
  - Indexing
  - Query structure alignment

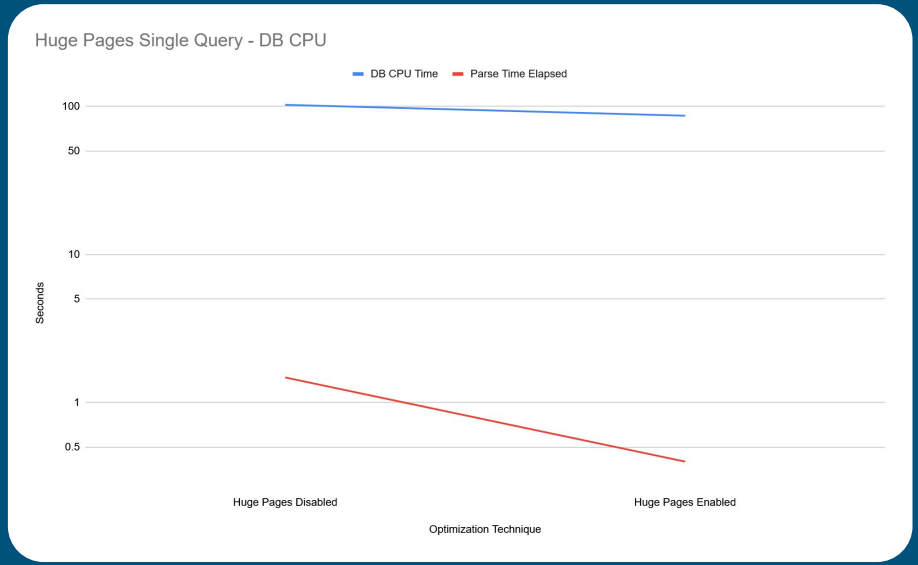
Testing & Validation

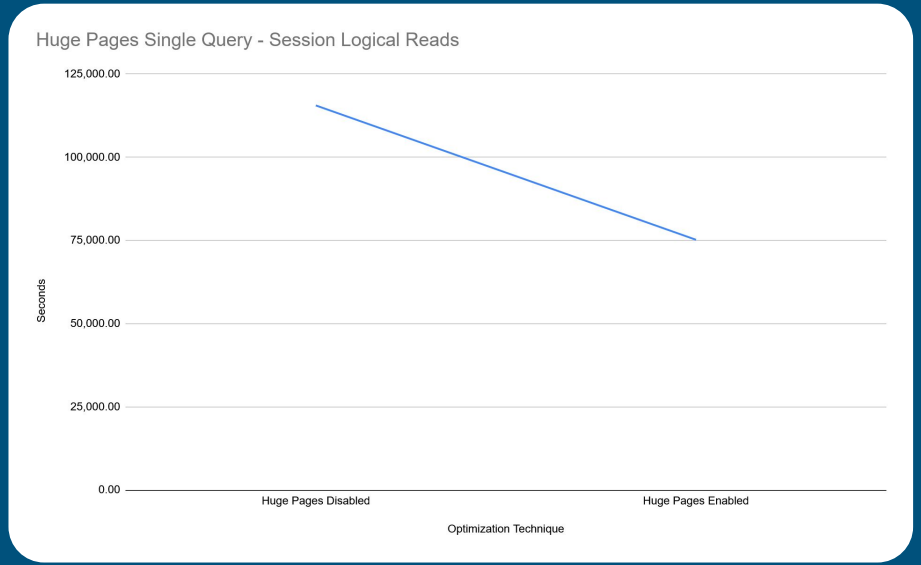
Example Query: Return all documents with keywords starting with "learn" (learning, learnable, etc.) by authors whose last name starts with S (Smith, Singh, Santos, etc.), sorted by most co-authors first.

Approx 791,000 records

```
    SELECT d.title,
           d.document_type,
           d.doi_id,
           DBMS_LOB.SUBSTR(d.abstract, 300, 1) AS abstract_preview,
           a.last_name,
           a.first_name,
           o.struct_name                       AS institution,
           k.keyword_text,
           (SELECT COUNT(*)
              FROM doc_author da2
             WHERE da2.document_key = d.document_key) AS coauthor_count
      FROM document     d
      JOIN doc_keyword dk ON d.document_key  = dk.document_key
      JOIN keyword      k  ON dk.keyword_key  = k.keyword_key
      JOIN doc_author   da ON d.document_key   = da.document_key
      JOIN author       a  ON da.author_key    = a.author_key
 LEFT JOIN author_organism ao ON da.document_key = ao.document_key
                             AND da.author_key   = ao.author_key
 LEFT JOIN organism     o  ON ao.organism_key  = o.organism_key
     WHERE k.keyword_text LIKE v_kw_prefixes(v_kw_idx)
       AND a.last_name    LIKE v_auth_prefixes(v_auth_idx)
     ORDER BY coauthor_count DESC, d.title;
```

Testing & Validation

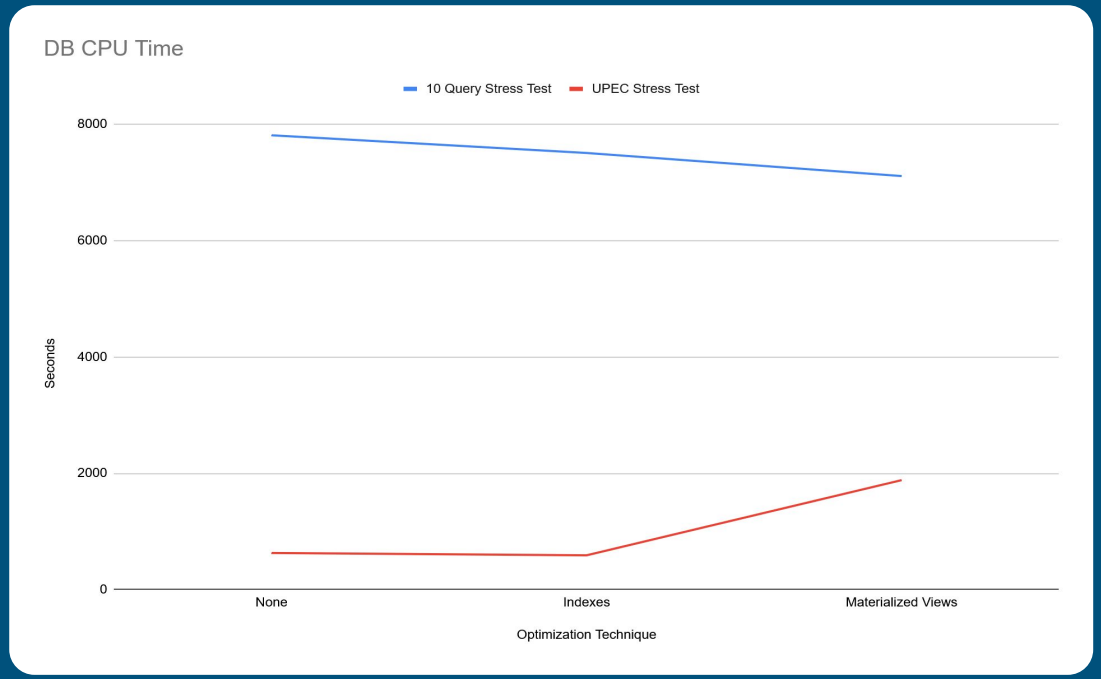




DB CPU Time	102.72	86.72	-15.58%
Parse Time Elapsed	1.48	0.4	-72.97%

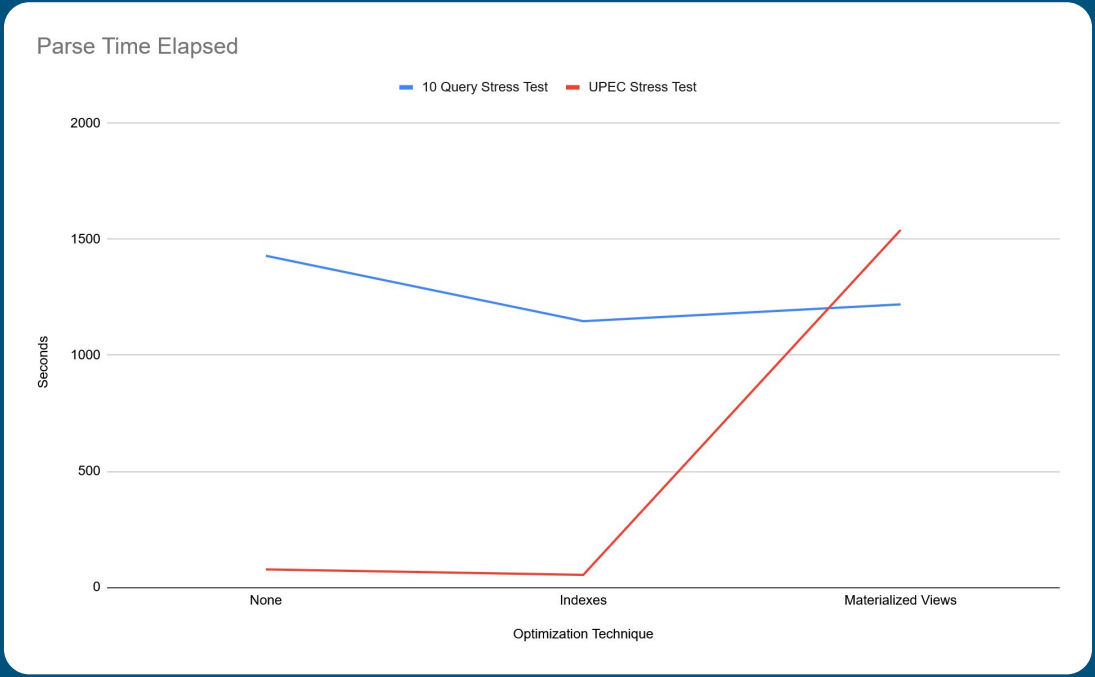
Session Logical Reads	115,554.00	75,202.00	-34.92%

Testing & Validation



10 Query Stress Test	MV: -8.97%	Index: -3.89%
UPEC Stress Test	MV: 198.45%	Index: -6.28%

Testing & Validation



10 Query Stress Test	MV: -14.66%	Index: -19.73%
UPEC Stress Test	MV: 1,896.55%	Index: -30.33%

Success Metrics

- General Requirements
- Industry Standard
- Transaction Processing Performance Council (TPC)

QUERY RESPONSE TIME

Avg DB time per call	Parse time elapsed/txn	Workarea mode
0.03s	4.48s	Optimal
Target: ≤ 0.5s per call	Target: ≤ 10s	All 8,424 sorts in memory — 0 spills

TRANSACTIONS PER SECOND (TPS)

SQL executions/txn	User calls/txn	User rollbacks/txn
169,199	3,672	0
Target: ≥ 100,000	Target: ≥ 2,000	Target: 0

UPTIME & RELIABILITY

User rollbacks	Multipass workarea executions
0	0
Target: 0 — no failures under load	Target: 0 — no memory pressure failures

ALL SUCCESS METRICS

Query response time			
Avg DB time per call	≤ 0.5s	0.03s	Pass
Parse time elapsed per txn	≤ 10s	4.48s	Pass
Physical reads per txn	≤ 5 blocks	0	Pass
Logical reads per txn	≤ 15,000 blocks	10,572	Pass
All sort operations in optimal memory mode	0 onepass / 0 multipass	8,424 optimal	Pass
Transactions per second (TPS)			
SQL executions per txn	≥ 100,000	169,199	Pass
User calls per txn	≥ 2,000	3,672	Pass
Session cursor cache hits per txn	≥ 150,000	166,559	Pass
Buffer cache hit ratio	≥ 95%	100%	Pass
db block changes per txn	≤ 2,000	1,447	Pass
Uptime & reliability			
User rollbacks	0	0	Pass
Multipass workarea executions (memory spills)	0	0	Pass
Onepass workarea executions	0	0	Pass
Sustained load across 1,000 sessions × 1,000 loops	No errors or failures	Clean run	Pass

Backup & Recovery Validation

Backup

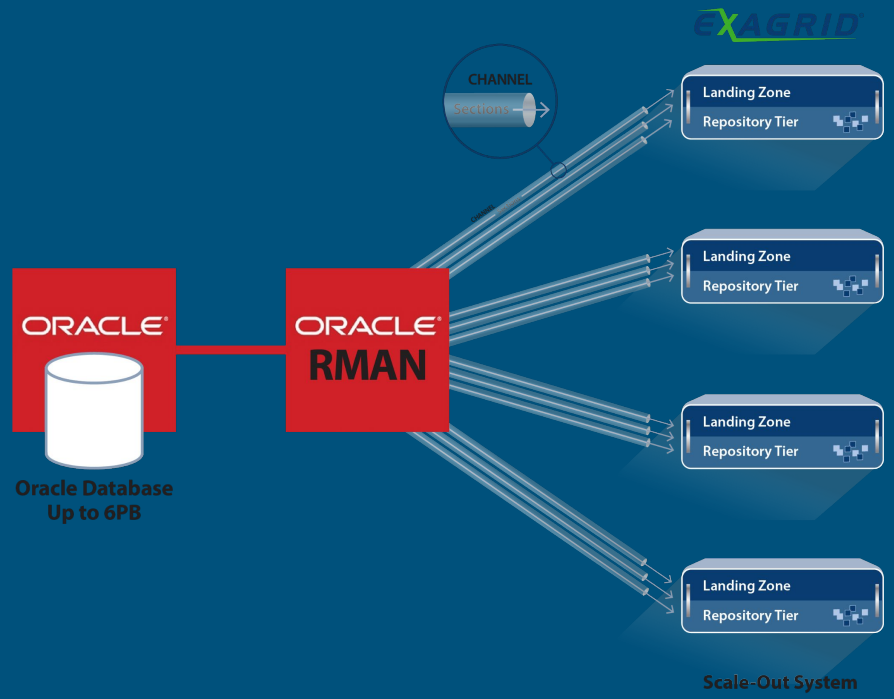
- RMAN full + incremental backups
- Includes redo logs & control files

Recovery

- RMAN & Data Recovery Advisor
- Detects and repairs failures
- Supports point-in-time recovery

Validation

- Backup restore tested successfully
- Database recovered after failure
- Ensures system reliability



Automated Performance Testing

- Run workload queries before changes
- Apply schema updates (indexes, MV, partitions)
- Re-run same queries
- Compare execution time
- Threshold:
- 10% slower → reject changes

```
[root@cosc416-ora-james Stress_Testing]# ./run_read_stress.sh 1000
Starting read stress test: 1000 parallel sessions
Start time: 2026-04-01 12:40:15
---
---
Read stress test complete.
End time: 2026-04-01 12:40:17
Sessions: 1000
Wall clock time: 0s

Per-session logs : read_stress_session_*.log
Consolidated log : ./logs/read_stress_consolidated.log
```

- Ensures no performance regression

Baseline Performance Review

Good

- Perfect In-Memory Sort
- Perfect Buffer Nowait
- Perfect Redo Nowait

Needs Improvement

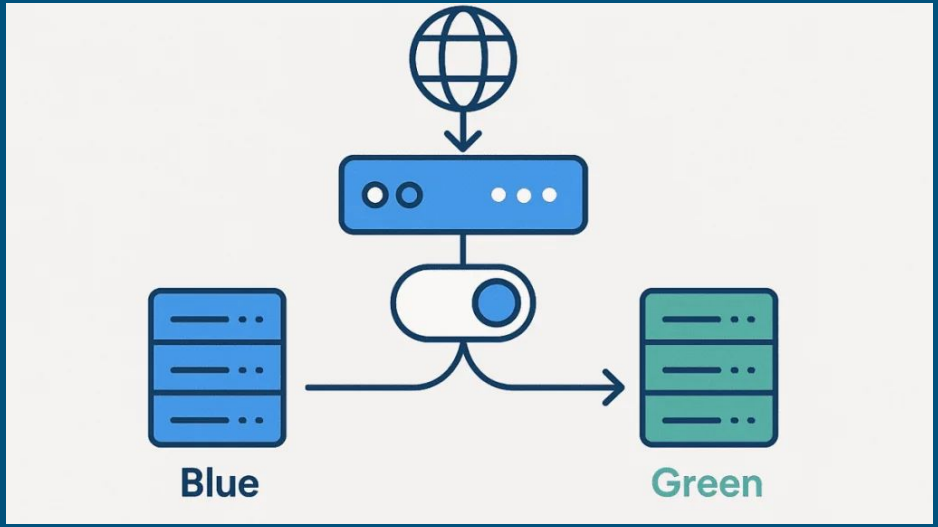
- Soft Parse below 95%
- Very Low Execute to Parse

Instance Efficiency Percentages (Target 100%)

Buffer Nowait %:	100.00	Redo NoWait %:	100.00
Buffer Hit %:	99.97	In-memory Sort %:	100.00
Library Hit %:	92.24	Soft Parse %:	92.65
Execute to Parse %:	34.35	Latch Hit %:	99.87
Parse CPU to Parse Elapsd %:	92.33	% Non-Parse CPU:	96.19
Flash Cache Hit %:	0.00		

Blue-Green Deployment for Databases

- Scheduled Writes only
- Load Balancer
- Distribute Traffic
- Rollback on errors



Post-Deployment Monitoring

- Monitor CPU, wait events (OEM)
  - CPU usage spikes (>80%)
  - High wait events
- Track system load (Active Sessions)
- Compare AWR snapshots (before vs after)
  - DB time increase (>10%)

Refactoring & Iteration

- Initial Issues:
  - Slow join-heavy queries
  - High resource usage
- Improvements:
  - Materialized views
  - Indexing on key columns
  - Reduced full table scans
- Result:
  - ~40% lower DB time
  - Reduced I/O (near 0 physical reads)
  - Lower CPU usage

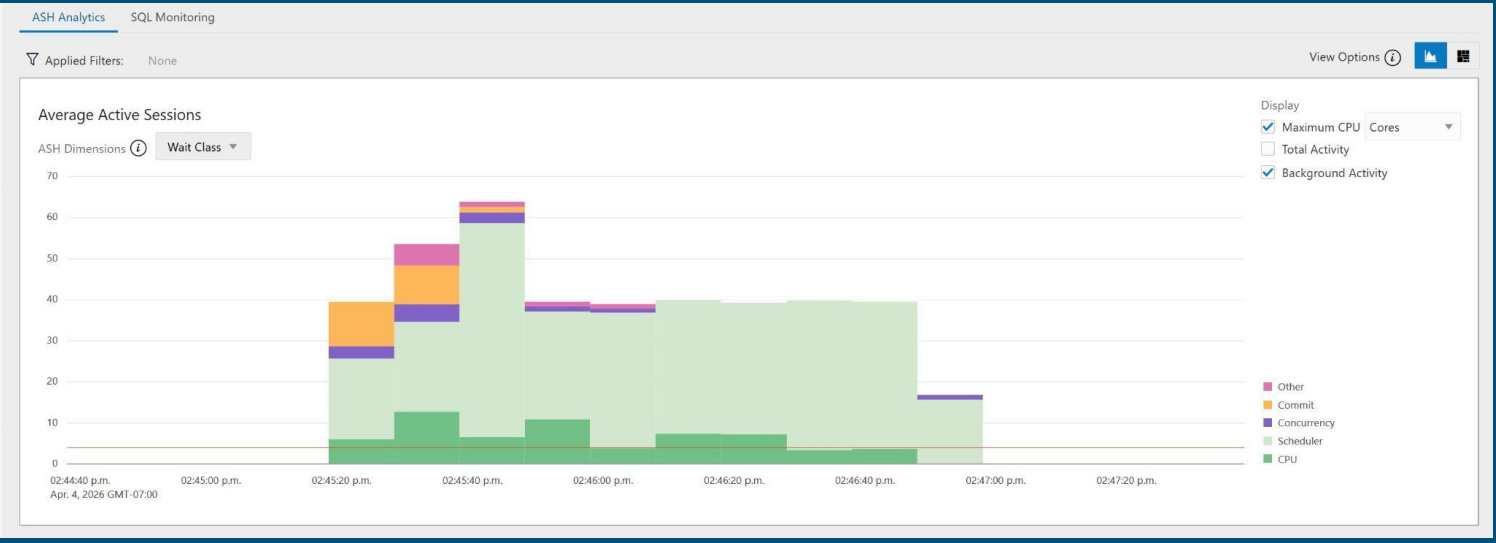
Operational Release & Transition

- Apply schema changes:
  - Indexes, partitions, materialized views
- Run workload validation script
- Compare performance (before vs after)
- Rollback if performance degrades
- Monitor system after deployment

Operational Runbooks

Common Issues & Actions

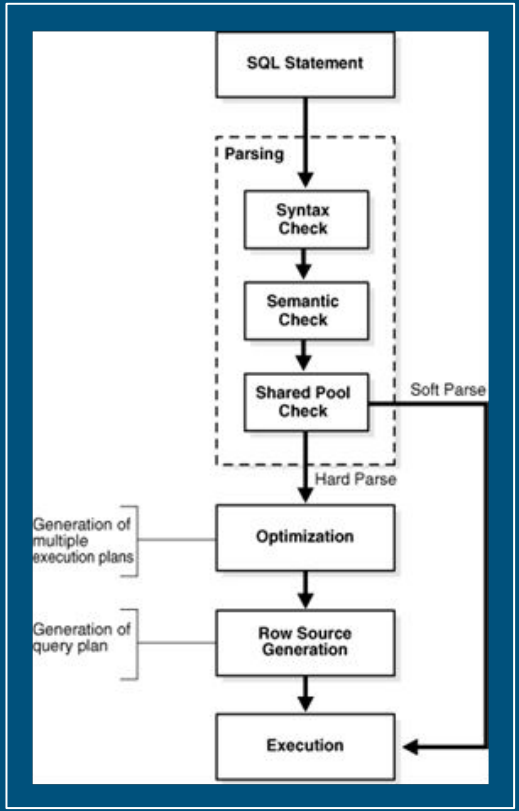
- High CPU (resmgr:cpu quantum)
  - Reduce sessions / optimize queries
- Log File Sync Waits
  - Reduce commit frequency
- Low Soft Parse Ratio (<95%)
  - Enable session caching / reuse SQL
- Full Table Scans
  - Add indexes (author_key, last_name)
- Materialized View Not Used
  - Check execution plan / refresh MV



Knowledge Transfer

Data Access Paths

- Keyword and Document Lookups
- Author Based Lookups
- Cross-Entity Joins
- Unfiltered Full-Table Aggregations

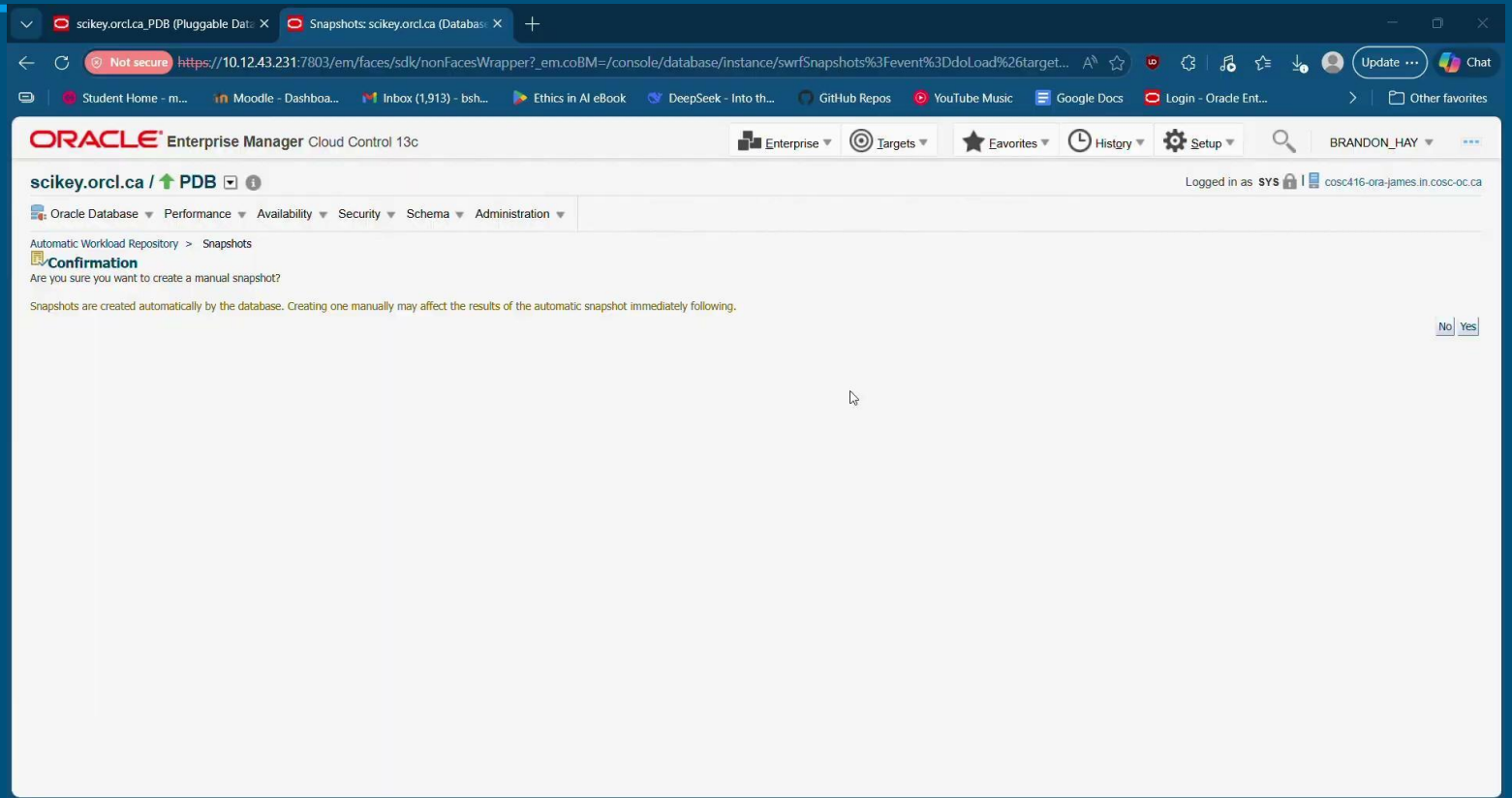


Schema Versioning

Github Repo: https://github.com/ToxicOstrich/OC_SciKey_COSC_416/tree/main

- Database Setup
  - Indexes
  - Partitions
  - Materialized Views
  - Data Manipulation Scripts
- Stress Test
  - Single Query
  - Many Query
- Documentation

Operational Release & Transition, Video Demo



Thanks to the Langara College Team

Their work was a valuable reference that allowed us to understand the project better early on

Reference:
T. E. Singson, D. C. Silva, M. C. Sandate, B. Bernabeo, K. E. Camota, and T. S. Sheriff, "SciKey" Langara College, Vancouver, BC, Canada, 2025. Available: https://github.com/santre8/SciKey

Any Questions?